

의존성 역전 원칙 위반과 해결

객체지향설계와 패턴
2026.03.24(Tue)



Workbook Lab #03 : InLab #5 DIP

- 여기에 보이는 해결방법은 최소한의 내용이라 완전하다고 할 수 없다.
- 사례는 처음에는 간단한 기능 테스트를 위해 만든 코드에서 조금씩 기능을 추가한 사례로 보여진다.
- C# 소스 프로그램이라 많이 생소할 수 있다.
- 사례 자체에서 S.O.L.I.D의 위반 사례가 있다.
- 해결 방법으로는 의존성을 외부에서 주입하는 방법 (Dependency Injection)이 가장 쉬운 내용이다.
 - DI를 쉽게 이해시키기 위해 사용된 예제들과 많이 유사

```
public class PersonRepository : IReadablePersonRepository, IWritablePersonRepository
{
    public void Save(Person person)
    {
        using (var db = new DbContext())
        {
            return db.People.Add(person);
        }
    }
    public void Save(IEnumerable<Person> people)
    {
        using (var db = new DbContext())
        {
            return db.People.AddRange(people);
        }
    }
}
```

```
    public Person Get(int id)
    {
        using (var db = new DbContext())
        {
            return db.People.GetById(id);
        }
    }
    public IEnumerable<Person> Get()
    {
        using (var db = new DbContext())
        {
            return db.People.ToEnumerable();
        }
    }
    public IEnumerable<Person> Get(Func<Person, bool> predicate)
    {
        using (var db = new DbContext())
        {
            return db.People.Where(p => predicate(p)).ToEnumerable();
        }
    }
}
```

지적되는 내용

1. 어떤 부분이 의존성 역전 원칙을 위반했나?

- PersonRepository가 구체적인 데이터베이스 연결 객체인 "DbContext를 직접 생성(new)" 하고 있기 때문이다.
- 강한 결합(Tight coupling)
 - ❖ DbContext가 아닌 다른 DB나 Mock 객체로 교체하기가 불가능하다.
- 제어권 상실
 - ❖ 상위 모듈이 하위 모듈(DbContext)의 생명주기를 직접 관리하므로 '역전'이 일어나지 않았다.
- SRP 위반
 - ❖ 저장소의 역할과 더불어 DB 연결 관리 책임까지 떠안고 있다.
 - ❖ 제일 처음 알게 된 잘못

해결

- 은근히 유용하면서 이후 외부 변경에 도움되는 방법
 - 1. 구체적인 클래스가 아닌 인터페이스(추상화)에 의존
 - 2. 생성자를 통해 의존성을 외부에서 주입 받음(DI)
- 2. 의존성 주입을 적용한 코드의 수정/변경
 - 외부에서 추상화된 인터페이스(혹은 추상 클래스)를 주입 받도록 수정해야 한다.
 - 보통 IDbContext 같은 인터페이스를 사용하거나, 프레임워크에 따라 컨텍스트 자체를 주입 받는다.

```
public class PersonRepository : IReadablePersonRepository, IWritablePersonRepository
{
    // <1> 구체적인 클래스가 아닌 인터페이스(추상화)에 의존
    private readonly IDbContext _db;

    // <2> 생성자를 통해 의존성을 외부에서 주입받음 (DI)
    public PersonRepository(IDbContext db)
    {
        _db = db;
    }

    public void Save(Person person)
    {
        _db.People.Add(person);
    }

    public void Save(IEnumerable<Person> people)
    {
        _db.People.AddRange(people);
    }
}
```

```
    public Person Get(int id)
    {
        return _db.People.GetById(id);
    }

    public IEnumerable<Person> Get()
    {
        return _db.People.ToEnumerable();
    }

    public IEnumerable<Person> Get(Func<Person, bool> predicate)
    {
        return _db.People.Where(predicate).ToEnumerable();
    }
}
```

```
public interface IDbContext : IDisposable
{
    IDbSet<Person> People { get; }
    int SaveChanges(); // 변경 사항을 확정하는 공통 메서드
}

// 데이터 집합에 대한 추상화 (엔티티 조작용)
public interface IDbSet<T> where T : class
{
    void Add(T entity);
    void AddRange(IEnumerable<T> entities);
    T GetById(int id);
    IEnumerable<T> ToEnumerable();
    IEnumerable<T> Where(Func<T, bool> predicate);
}
```

앞서 수정한 PersonRepository가 의존할 IDbContext 인터페이스의 대략적인 예시이다.

개선 사항

3. 무엇이 개선되었나 ?

- 유연성(flexibility)
 - ❖ 이제 PersonRepository를 고치지 않고 SQL Server, MySQL, 혹은 테스트용 메모리 DB로 교체할 수 있다.
- 테스트 용이성(easy to test)
 - ❖ 단위 테스트 수행할 때 실제 DB 없이 가짜(Mock) IDbContext를 넣어 로직만 검증할 수 있다.
- 관심사 분리 *
 - ❖ DB 연결을 열고 닫는 책임은 이 클래스(PersonRepository)의 외부(DI 컨테이너 등)로 옮겨졌다.

결론

- DIP의 핵심은 상위 모듈(PersonRepository)이 하위 모듈(DbContext 실체)의 구체적인 구현을 몰라도 동작하게 만드는 것이다.
- 인터페이스에는 데이터 저장소로서 필요한 최소한의 행위만 정의한다.